```
from flask import render_template, flash, redirect, url_for, request
from app import app, db
from forms import LoginForm, ParamsForm
from models import User, Transformer
from flask_login import current_user, login_user, logout_user, login_required
from werkzeug.urls import url_parse

@app.route('/login', methods=['GET', 'POST'])
def login():
    if current_user.is_authenticated:
        return redirect(url_for('index'))
    form = LoginForm()
    if form.validate_on_submit():
        user = User.query.filter_by(username=form.username.data).first()
        if user is None or not user.check_password(form.password.data):
            flash('无效的用户名或密码')
            return redirect(url_for('login'))
        login_user(user, remember=form.remember_me.data)
        next_page = request.args.get('next')
        if not next_page or url_parse(next_page).netloc != '':
            next_page = url_for('index')
        return redirect(next_page)
    return render_template('login.html', title='登录', form=form)

@app.route('/logout')
def logout():
    logout_user()
    return redirect(url_for('index'))

    @app.route('/input_params', methods=['GET', 'POST'])
    @login_required
    def input_params():
        form = ParamsForm()
        if form.validate_on_submit():
            transformer = Transformer(
                rated_current=form.rated_current.data,
                rated_voltage=form.rated_voltage.data,
                short_circuit_impedance=form.short_circuit_impedance.data,
                rated_power=form.rated_power.data,
                transformer_type=form.transformer_type.data,
                voltage_adjustment=form.voltage_adjustment.data
            )
            db.session.add(transformer)
            db.session.commit()
```

```
                flash('参数已提交')
                return redirect(url_for('network_model'))
            elif request.method == 'POST' and 'reset' in request.form:
                form.rated_current.data = ''
                form.rated_voltage.data = ''
                form.short_circuit_impedance.data = ''
                form.rated_power.data = ''
                form.transformer_type.data = ''
                form.voltage_adjustment.data = ''
                flash('参数已重置')
                return render_template('input_params.html', title='参数输入', form=form)
            return render_template('input_params.html', title='参数输入', form=form)
        from flask_wtf import FlaskForm
        from wtforms import FloatField, SelectField, SubmitField
        from wtforms.validators import DataRequired
        class ParamsForm(FlaskForm):
            rated_current = FloatField('额定电流（A）', validators=[DataRequired()])
            rated_voltage = FloatField('额定电　　压（KV）', validators=[DataRequired()])
            short_circuit_impedance = FloatField('短路阻抗（pu）', validators=[DataRequired()])
            rated_power = FloatField('额定功率（MVA）', validators=[DataRequired()])
            transformer_type = SelectField('变压器类型', choices=[('油浸式变压器', '油浸式变压
器'),('干式变压器', '干式变压器'), ('隔离变压器', '隔离变压器'),('自耦变压器', '自耦变压器'),('
分配变压器', '分配变压器'),('三相变压器', '三相变压器'), ('动态变压器', '动态变压器'),('共变
压器', '共变压器')],
            validators=[DataRequired()])
            voltage_adjustment = FloatField('调整端头电压值（%）', validators=[DataRequired()])
            submit = SubmitField('完成输入')
            reset = SubmitField('重置数据')

# 更新路由，处理 POST 请求并获取用户选择的连接方式
@app.route('/network_model', methods=['GET', 'POST'])
@login_required
def network_model():
    if request.method == 'POST':
        connection_type = request.form.get('connection_type')
        transformer = Transformer.query.order_by(Transformer.id.desc()).first()
        transformer.connection_type = connection_type
        db.session.commit()
        flash('电气网络模型建立')

        # 重置模型
        return redirect(url_for('assessment'))
    connection_type = SelectField('连接方式', choices=[('star', '星形连接'), ('triangle', '三角形
连接')], validators=[DataRequired()])
```

```
89
90  # 在 network_model 路由中保存用户选择的连接方式
91  @app.route('/network_model', methods=['GET', 'POST'])
92  @login_required
93  def network_model():
94      form = ParamsForm()
95      if request.method == 'POST' and form.validate_on_submit():
96          connection_type = form.connection_type.data
97          transformer = Transformer.query.order_by(Transformer.id.desc()).first()
98          transformer.connection_type = connection_type
99          db.session.commit()
100         flash('电气网络模型已创建')
101         return redirect(url_for('assessment'))
102     return render_template('network_model.html', title='电气网络建模', form=form)
103
104     @app.route('/assessment', methods=['GET', 'POST'    ])
105 @login_required
106 def assessment():
107     if request.method == 'POST':
108         short_circuit_condition = request.form.get('short_circuit_condition')
109         short_circuit_current = calculate_short_circuit_current()
110         stability = evaluate_stability()
111         safety = evaluate_safety()
112     calculation_method = request.form.get('calculation_method')
113     # 根据短路条件和计算方式执行相应的操作
114     if short_circuit_condition == '单相短路':
115         # 执行单相短路计算
116         result = calculate_single_phase_short_circuit(calculation_method)
117     elif short_circuit_condition == '两相短路':
118         # 执行两相短路计算
119         result = calculate_two_phase_short_circuit(calculation_method)
120     # 其他短路条件类似
121
122     # 处理完后返回相应的响应或重定向到结果页面
123     return render_template('assessment_result.html', result=result)
124
125     # 根据短路条件和计算方式执行相应的操作
126         if short_circuit_condition and calculation_method:
127             # 根据选择的计算方法进行相应的短路电流计算
128             if calculation_method == '等效电路模型计算':
129                 short_circuit_current=
130 calculate_short_circuit_equivalent(short_circuit_condition)
131             elif calculation_method == '短路阻抗法计算':
132                 short_circuit_current=
```

```
calculate_short_circuit_impedance(short_circuit_condition)
                elif calculation_method == '有限元法计算':
                    short_circuit_current=
calculate_short_circuit_finite_element(short_circuit_condition)
                else:
                    flash('选择的计算方法不支持')
                    return redirect(url_for('assessment'))

                # 执行稳定性评估和安全性评估
                stability = evaluate_stability(short_circuit_condition)
                safety = evaluate_safety(short_circuit_condition)

                # 更新数据库中的变压器信息
                transformer = Transformer.query.order_by(Transformer.id.desc()).first()
                transformer.short_circuit_condition = short_circuit_condition
                transformer.short_circuit_current = short_circuit_current
                transformer.stability = stability
                transformer.safety = safety
                transformer.result = "分析短路电压完成"
                db.session.commit()
                flash('稳定性评估已完成')
                return redirect(url_for('results'))
        return render_template('assessment.html', title='安全性评估')

# 结果输出
@app.route('/results')
@login_required
def results():
    transformers = Transformer.query.all()
    return render_template('results.html', title='结果输出', transformers=transformers)
# 清空结果
@app.route('/results/clear')
@login_required
def clear_results():
    Transformer.query.delete()
    db.session.commit()
    flash('所有结果已清空')
    return redirect(url_for('results'))
# 导出结果
@app.route('/results/export')
@login_required
def export_results():
    # 导出
    flash('结果已导出')
```

```
177         return redirect(url_for('results'))
178     # 打印结果
179     @app.route('/results/print')
180     @login_required
181     def print_results():
182         # 打印
183         flash('结果已打印')
184         return redirect(url_for('results'))
185
186     # 退出
187     @app.route('/logout')
188     def logout():
189         logout_user()
190         return redirect(url_for('index'))
```